California Housing Price Prediction — Linear Regression in Python and R

Predicting median district house values from socioeconomic and geographic features using linear regression, implemented in parallel in Python and R.

Jibril Hussaini | Python · scikit-learn · pandas · matplotlib · R · caret · dplyr · ggplot2

Overview

This project builds a linear regression model to predict the median house value (MedHouseVal) of California districts using eight socioeconomic and geographic features from the California Housing dataset. The same end-to-end workflow is implemented twice — once in Python and once in R — to compare how the two languages handle each stage of the pipeline and whether they converge on the same results.

Goals

Deliver a clean, well-evaluated regression pipeline in both Python and R, and quantify how closely the two implementations agree on predictive accuracy.

Objectives

- Load the California Housing dataset and inspect the first five rows; identify and impute any missing values with column means.
- Scale all features with StandardScaler (Python) or scale() (R) to put them on a common footing before modelling.
- Split the data 80/20 into training and test sets, train a linear regression model on the training set, and generate predictions on the test set.
- Evaluate each model with R-squared and mean squared error, and display the learned feature weights and bias term.
- Visualise predicted versus actual house prices to assess model fit and compare results between the two languages.

Approach

Each notebook follows an identical 12-step pipeline: data loading, inspection, missing-value imputation, feature scaling, train/test splitting, model training, prediction, evaluation, weight display, and visualisation. Implementing the same steps independently in Python (scikit-learn) and R (caret) provides a controlled comparison of the two ecosystems on the same task and dataset.

Outcomes

- Python linear regression achieved R-squared of 0.576 and MSE of 0.556 on the held-out test set.
- R linear regression achieved R-squared of 0.592 and MSE of 0.564 on the held-out test set.
- Both implementations converged on consistent results; the small differences stem from differing random-seed behaviour in the train/test partition routines.

- The parallel implementation confirmed that scikit-learn and caret expose equivalent regression interfaces and produce statistically equivalent models on the same data.